Electronic Voting Machine Using ESP32

By: Vignesh SB 22MID0190
Sanjay SM 22MID0189
Adit Ranganathan Suresh 22MID0197

Introduction

The rapid evolution of digital technologies has paved the way for innovative solutions in many fields, including the electoral process. Electronic voting machines aim to improve the speed, accuracy, and reliability of vote counting while minimizing human error. In this project, we use the ESP32 module as the heart of the system to control various peripheral components and provide a seamless user interface. The following sections provide a detailed introduction to each component used, explaining their roles and significance within the voting machine design.

Components Overview

ESP32 Module

The ESP32 is a highly integrated microcontroller renowned for its performance and connectivity, making it an ideal core for IoT applications, including electronic voting machines.

•Processor and Architecture:
The ESP32 is equipped with a dual-core Tensilica LX6 processor, enabling it to handle multiple tasks concurrently. With clock speeds up to 240 MHz, it provides ample processing power for real-time operations such as user input handling, data processing, and communication with peripheral devices.

•Wireless Connectivity:
Integrated Wi-Fi and Bluetooth (including BLE) capabilities allow the ESP32 to communicate wirelessly. In the context of a voting machine, this can be used for:

- •Transmitting votes to a central server.
- •Receiving system updates or configuration changes.
- •Implementing remote monitoring and diagnostics.

•Peripheral Interfaces:
The ESP32 offers a variety of I/O interfaces including:

- •GPIO Pins: For connecting sensors, buttons, and other digital devices.
- •I2C, SPI, UART: Communication protocols that simplify the connection of external devices like LCDs and other modules.
- •Analog Inputs: Useful for reading variable signals if required by the application.

•Low Power Consumption:
Its power efficiency makes it suitable for battery-operated systems or scenarios where energy conservation is critical. Features such as deep sleep mode further enhance its suitability for long-term deployments.

•Development and Programming:
Using the Arduino IDE, developers benefit from an extensive library ecosystem, which simplifies the process of coding complex functionalities. Libraries for handling Wi-Fi, Bluetooth, and various peripherals are readily available, making the ESP32 a flexible choice for prototype development and eventual production.

LCD 16x2 I2C Display

The LCD 16x2 I2C display is a compact and user-friendly output module that provides a clear interface for presenting information, such as voting options or status messages.

- Display Specifications:
  - Screen Size and Resolution:
    The display consists of 2 rows with 16 characters per row, which is adequate for conveying concise messages and prompts.
  - Character Controller:
    It typically uses the HD44780 controller, which has become the industry standard for character LCDs due to its reliability and ease of programming.
- I2C Communication Interface:
  - Reduced Wiring Complexity:
    The I2C (Inter-Integrated Circuit) interface requires only two data lines (SDA for data and SCL for the clock), in addition to power and ground. This significantly reduces the number of connections required compared to a parallel interface, thereby simplifying circuit design and freeing up additional pins on the ESP32.
  - I2C Module:
    Often, these displays include an I2C backpack (based on chips like the PCF8574), which converts the parallel communication required by the HD44780 controller into an I2C interface. This allows for easy integration with microcontrollers like the ESP32.
- Library Support and Functionality:
  - Arduino Libraries:
    Libraries such as `LiquidCrystal_I2C` provide user-friendly functions for initializing the display, sending text, and controlling backlight settings. This abstraction layer allows developers to quickly implement and modify the user interface.
  - User Interface Design:
    The display can be used to show menus, instructions, candidate names, and confirmation messages. For example, in an electronic voting machine, it can provide feedback such as “Vote Recorded” or “Select Option,” ensuring clear communication with the voter.

Push Buttons

Push buttons are an essential part of the user input system for the electronic voting machine. They allow voters to make selections in a straightforward and user-friendly manner. In this project, multiple push buttons are integrated to represent different voting options or to navigate through menus. The simplicity and reliability of mechanical push buttons help reduce potential points of failure, ensuring that each vote is accurately recorded.

Jumper Wires

Jumper wires are used to make the necessary electrical connections between the ESP32, LCD, push buttons, and other components on the circuit board. They are flexible and available in various lengths, which is ideal for prototyping on a breadboard. Their ease of use helps in quickly setting up and modifying the circuit, thereby facilitating rapid development and troubleshooting of the electronic voting system.

Arduino IDE

The Arduino Integrated Development Environment (IDE) is the platform used for coding and programming the ESP32. This open-source environment supports a wide range of microcontrollers, including the ESP32, and provides a user-friendly interface for writing, compiling, and uploading code. With its vast community and abundant libraries, the Arduino IDE simplifies the development process, making it easier to implement functionalities such as button debouncing, LCD control, and wireless communication.

Micro USB to USB A Cable

The micro USB to USB A cable is utilized for programming the ESP32 and for providing power during development and testing. This cable facilitates a reliable connection between the microcontroller and a computer, allowing developers to upload firmware via the Arduino IDE. The USB interface also serves as a power source, ensuring that the ESP32 and connected peripherals receive the necessary voltage for operation during initial stages and debugging.

Breadboard

A breadboard is a prototyping platform that allows for the easy assembly and modification of electronic circuits without soldering. In this project, the breadboard is used to lay out the connections between the ESP32, LCD, push buttons, and other components. Its reusable design makes it an invaluable tool during the development phase, enabling quick adjustments and testing of the circuit design before moving on to a more permanent solution.

Overall System Architecture

At the heart of the system is the ESP32 module, which is responsible for reading button inputs, processing vote counts, and controlling the LCD display via the I2C interface. The code, written in the Arduino IDE, ties together hardware components (push buttons, LCD, and supporting wiring) to create a real-time voting application.

ESP32 Module

Role in the Project:

•Central Controller: The ESP32 acts as the central microcontroller that reads inputs from push buttons, maintains vote counters, and updates the LCD display.

•Debouncing and Processing: It handles the debouncing logic for each button to ensure that a single press is counted only once. This is critical because mechanical buttons can generate spurious signals when pressed.

Code Insights:

•Pin Configuration:

```
// Set candidate and extra buttons as input with internal pull-ups.
pinMode(candidate1ButtonPin, INPUT_PULLUP);
pinMode(candidate2ButtonPin, INPUT_PULLUP);
pinMode(candidate3ButtonPin, INPUT_PULLUP);
pinMode(resetButtonPin, INPUT_PULLUP);
pinMode(showResultsButtonPin, INPUT_PULLUP);
```

The ESP32 initializes its GPIO pins as inputs using internal pull-up resistors. This means that when the button is not pressed, the pin reads HIGH, and when pressed, it reads LOW.

•Debouncing Logic:
For each button (candidate, reset, and show results), the code tracks the last known state and the time since the last change. Only if the new state remains stable for at least `debounceDelay` (50 ms) does the code accept the press as valid.

```
if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading == LOW) {
        voteCount++;  // For candidate buttons.
        // ...
    }
}
```

•Real-Time Processing:
Inside the `loop()`, the ESP32 continuously checks for button presses using helper functions like `checkButton()`, `checkResetButton()`, and `checkShowResultsButton()`. This design ensures that user actions are immediately processed and reflected on the LCD.

LCD 16x2 I2C Display

Role in the Project:

•User Interface: The LCD serves as the primary means of communicating with the user. It displays the welcome message, voting instructions, vote confirmations, and final vote counts.

•Feedback Mechanism: It informs the voter when their vote has been registered and later shows the overall results when requested.

Code Insights:

•Initialization and Setup:

```
// Initialize the LCD at I2C address 0x27, 16 columns, and 2 rows.
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
    lcd.init();
    lcd.backlight();
    // Initial welcome message.
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Voting Machine");
    lcd.setCursor(0, 1);
    lcd.print("Initializing...");
    delay(2000);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Voting in progress...");
}
```

The display is set up using the `LiquidCrystal_I2C` library.
The `lcd.init()` function initializes communication, and `lcd.backlight()` ensures that the display is visible. Messages are then sent to the LCD using commands like `lcd.print()`.

•Vote Confirmation and Results Display:
When a candidate button is pressed, the code calls `showVoteConfirmation()`, which clears the LCD and displays a brief confirmation message:

```
void showVoteConfirmation(String candidateName) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Voted: ");
    lcd.print(candidateName);
    delay(1500);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Voting in progress...");
}
```

Similarly, when the "show results" button is pressed, the function `updateDisplay()` is invoked to show the final vote counts:

```
void updateDisplay() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("C1:");
    lcd.print(candidate1Votes);
    lcd.print(" C2:");
    lcd.print(candidate2Votes);
    lcd.setCursor(0, 1);
    lcd.print("C3:");
    lcd.print(candidate3Votes);
}
```

Push Buttons and Their Roles

Candidate Buttons:

- Purpose: Three separate buttons represent the vote options for three different candidates.
- Implementation: Each button press triggers the `checkButton()` function, where the code handles debouncing and increments the appropriate candidate's vote counter.

Reset Button:

- Purpose: Allows the system to reset the election, clearing all vote counts and restarting the process.
- Implementation: The `checkResetButton()` function monitors this button. When pressed, it resets vote counters and updates the display to indicate the reset.

Show Results Button:

- Purpose: When pressed, the election is marked as ended, and the final vote tallies are displayed on the LCD.
- Implementation: Managed by `checkShowResultsButton()`, which sets a flag (`electionEnded`) to prevent further vote counting and calls `updateDisplay()`.

Additional Components in the Hardware Setup

Jumper Wires:

- Purpose: Connect the ESP32 to the LCD and buttons, facilitating communication and power delivery.
- Usage: In the physical build, jumper wires ensure that signals from the push buttons reach the corresponding GPIO pins and that the I2C signals from the LCD are properly routed.

Breadboard:

- Purpose: Provides a platform for assembling the circuit without soldering.
- Usage: The ESP32, LCD, and buttons are all mounted on a breadboard during prototyping. This setup allows for quick adjustments and testing before finalizing the circuit design.

Micro USB to USB A Cable and Arduino IDE:

•Purpose: The cable is used for power and programming the ESP32, while the Arduino IDE is used for writing and uploading the code.

•Usage: The IDE facilitates rapid development with its supportive libraries, enabling the integration of hardware components through straightforward code functions.

Electronic Voting Machine Connection

1. Power Distribution

3.3V Power Supply:

•The ESP32 supplies a regulated 3.3V, which is distributed via the breadboard's positive rail.

•The LCD I2C display is powered from the 3.3V rail, ensuring consistent voltage levels across all components.

•The common ground is established by connecting one of the ESP32’s GND pins to the breadboard's negative rail, providing a uniform reference for all components.

2. LCD 16x2 I2C Display Connections

The LCD module is equipped with an I2C interface that simplifies wiring by using only four connections:

•VCC: Connected to the 3.3V power rail on the breadboard.

•GND: Connected to the common ground rail.

•SDA (Serial Data): Connected via a jumper wire to the ESP32’s default SDA pin (commonly GPIO 21).

•SCL (Serial Clock): Connected via a jumper wire to the ESP32’s default SCL pin (commonly GPIO 22).

This configuration enables efficient communication between the ESP32 and the LCD, allowing for real-time display updates, such as vote confirmations and final results.

3. Push Buttons Wiring

The system utilizes five push buttons, each configured with the ESP32’s internal pull-up resistors. The buttons are distributed as follows:

•Candidate Vote Buttons (3):

•Candidate 1 Button: Connected to GPIO 13.

•Candidate 2 Button: Connected to GPIO 14.

•Candidate 3 Button: Connected to GPIO 27.

- Control Buttons (2):
  - Reset Button: Connected to GPIO 32.
  - Show Results Button: Connected to GPIO 33.

Wiring Details:

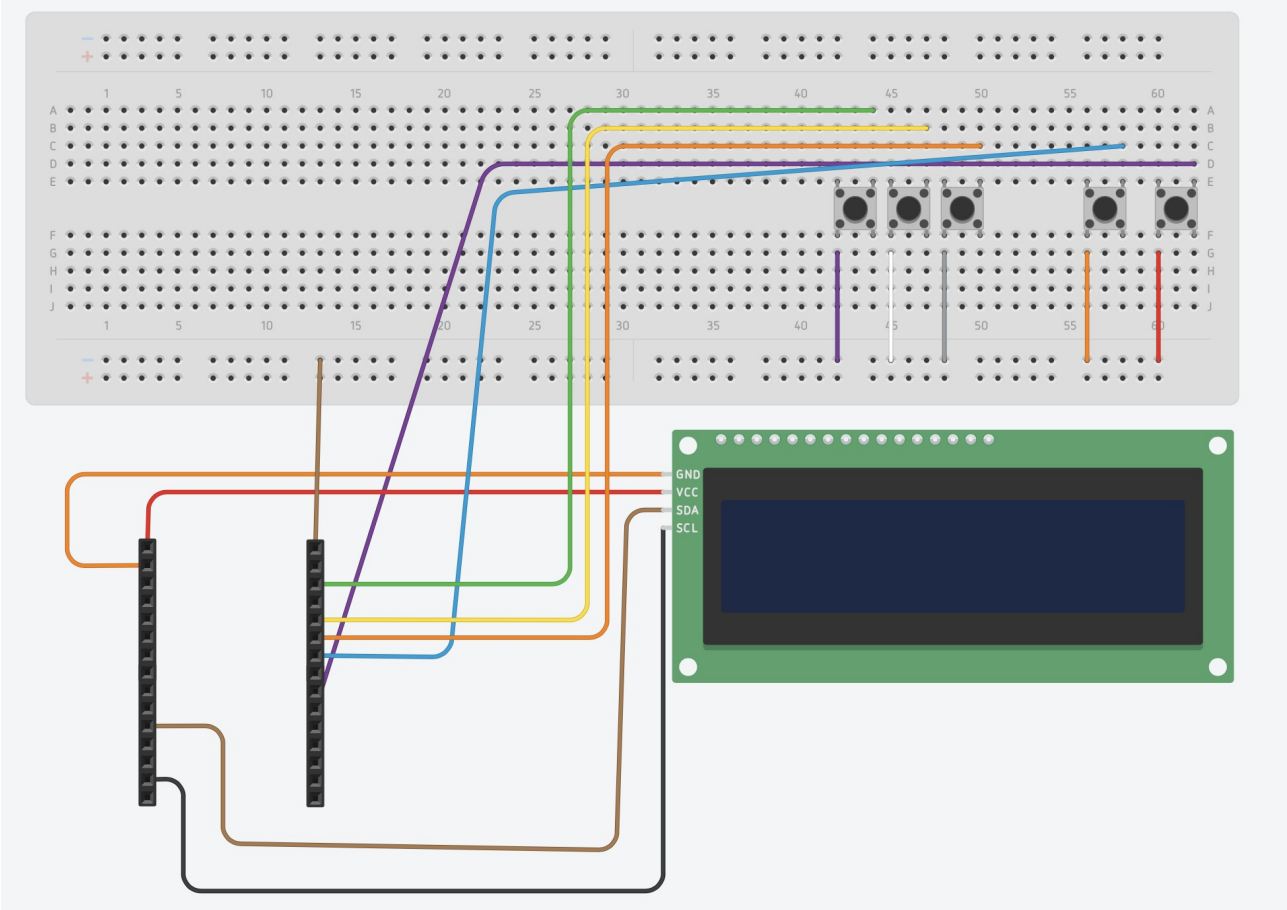
- One terminal of each button is connected to its respective GPIO pin on the ESP32.
- The opposite terminal of every button is connected to the ground rail on the breadboard.
- With the internal pull-up enabled, each button registers a HIGH state when unpressed. Pressing a button connects the GPIO pin directly to ground, registering a LOW signal, which is processed by the code for vote counting or control actions.

4. Breadboard Layout

The prototyping breadboard is organized as follows:

- Power Rails:
  - The positive rail is fed with 3.3V from the ESP32.
  - The ground rail is connected to the ESP32's ground pin.
- Component Placement:
  - LCD Display:
    Positioned so that its four pins align with the power rails and the ESP32's I2C lines (SDA and SCL).
  - Push Buttons:
    Each button is inserted into the breadboard such that one side connects to a designated ESP32 GPIO pin while the opposite side connects to the common ground rail.

This systematic layout ensures that all components receive stable power and proper signal routing, thereby enhancing the reliability and performance of the voting machine.



Wiring Diagram

Source Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Initialize the LCD at I2C address 0x27, 16 columns, and 2 rows.
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Define GPIO pins for the three candidate buttons
const int candidate1ButtonPin = 13; // Candidate 1
const int candidate2ButtonPin = 14; // Candidate 2
const int candidate3ButtonPin = 27; // Candidate 3

// Define GPIO pins for the reset and show results buttons
const int resetButtonPin = 32;
const int showResultsButtonPin = 33;

// Vote counters for each candidate
int candidate1Votes = 0;
int candidate2Votes = 0;
int candidate3Votes = 0;

// Variables for candidate button debouncing
bool lastButtonState1 = HIGH;
bool lastButtonState2 = HIGH;
```

```
bool lastButtonState3 = HIGH;
unsigned long lastDebounceTime1 = 0;
unsigned long lastDebounceTime2 = 0;
unsigned long lastDebounceTime3 = 0;

// Variables for reset and show results button debouncing
bool lastResetButtonState = HIGH;
bool lastShowResultsButtonState = HIGH;
unsigned long lastResetDebounceTime = 0;
unsigned long lastShowResultsDebounceTime = 0;

const unsigned long debounceDelay = 50;  // milliseconds

// Election state: false = voting in progress, true = election ended (results shown)
bool electionEnded = false;

void setup() {
  Serial.begin(115200);

  // Set up candidate buttons with internal pull-ups
  pinMode(candidate1ButtonPin, INPUT_PULLUP);
  pinMode(candidate2ButtonPin, INPUT_PULLUP);
  pinMode(candidate3ButtonPin, INPUT_PULLUP);

  // Set up reset and show results buttons with internal pull-ups
  pinMode(resetButtonPin, INPUT_PULLUP);
  pinMode(showResultsButtonPin, INPUT_PULLUP);

  // Initialize the LCD and turn on the backlight
  lcd.init();
  lcd.backlight();

  // Display welcome message and start election
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Voting Machine");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");
  delay(2000);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Voting in progress...");
}

void loop() {
  // Check extra buttons first
  checkResetButton();
  checkShowResultsButton();

  // Only process candidate votes if election is active
  if (!electionEnded) {
```

```
    checkButton(candidate1ButtonPin, candidate1Votes, lastButtonState1, lastDebounceTime1,
"Can: 1");
    checkButton(candidate2ButtonPin, candidate2Votes, lastButtonState2, lastDebounceTime2,
"Can: 2");
    checkButton(candidate3ButtonPin, candidate3Votes, lastButtonState3, lastDebounceTime3,
"Can: 3");
  }

  delay(10);  // Brief delay to reduce CPU usage
}

// Checks a candidate button press, increments vote count, and shows a brief vote confirmation.
void checkButton(int pin, int &voteCount, bool &lastButtonState, unsigned long
&lastDebounceTime, String candidateName) {
  int reading = digitalRead(pin);

  // If election ended, ignore candidate votes.
  if (electionEnded) {
    lastButtonState = reading;
    return;
  }

  // Reset debounce timer if the reading changes.
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  // If the reading is stable past the debounce delay, register the press.
  if ((millis() - lastDebounceTime) > debounceDelay) {
    // Button pressed (LOW due to pull-up)
    if (reading == LOW) {
      voteCount++;  // Increment vote count
      Serial.print(candidateName);
      Serial.print(" vote count: ");
      Serial.println(voteCount);
      showVoteConfirmation(candidateName);
      delay(200); // Prevent multiple counts from one press.
    }
  }

  lastButtonState = reading;
}

// Displays a vote confirmation message for 1.5 seconds.
void showVoteConfirmation(String candidateName) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Voted: ");
  lcd.print(candidateName);
  delay(1500);
  lcd.clear();
  lcd.setCursor(0, 0);
```

```
  lcd.print("Voting in progress...");
}

// Checks the reset button; when pressed, it resets all vote counts and restarts the election.
void checkResetButton() {
  int reading = digitalRead(resetButtonPin);
  if (reading != lastResetButtonState) {
    lastResetDebounceTime = millis();
  }
  if ((millis() - lastResetDebounceTime) > debounceDelay) {
    if (reading == LOW) { // Button pressed
      candidate1Votes = 0;
      candidate2Votes = 0;
      candidate3Votes = 0;
      electionEnded = false;
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Election Reset");
      delay(1500);
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Voting in progress...");
    }
  }
  lastResetButtonState = reading;
}

// Checks the show results button; when pressed, it ends the election and displays overall results.
void checkShowResultsButton() {
  int reading = digitalRead(showResultsButtonPin);
  if (reading != lastShowResultsButtonState) {
    lastShowResultsDebounceTime = millis();
  }
  if ((millis() - lastShowResultsButtonState) > debounceDelay) {
    if (reading == LOW) { // Button pressed
      electionEnded = true;
      updateDisplay();
    }
  }
  lastShowResultsButtonState = reading;
}

// Displays the overall vote counts on the LCD.
void updateDisplay() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("C1:");
  lcd.print(candidate1Votes);
  lcd.print(" C2:");
  lcd.print(candidate2Votes);
  lcd.setCursor(0, 1);
  lcd.print("C3:");
```

```
  lcd.print(candidate3Votes);
}
```

Output

When the electronic voting machine is powered on and the code is executed, the following sequence of outputs and events occur:

1. Initialization Phase:
   - LCD Display:
     Upon startup, the LCD clears any previous data and displays an introductory message:
     - First Line: "Voting Machine"
     - Second Line: "Initializing..."
   - This message remains for a short delay (approximately 2 seconds) to inform the user that the system is booting up.
2. Voting in Progress:
   - After initialization, the LCD updates to show:
     - First Line: "Voting in progress..."
   - At this stage, the system is actively monitoring for button presses.
   - Candidate Buttons:
     When a candidate button is pressed (with a brief debounce delay), the corresponding vote count is incremented, and the LCD temporarily displays a confirmation message:
     - Example Message: "Voted: Can: 1" (for candidate 1)
   - This confirmation message is shown for approximately 1.5 seconds before reverting to the "Voting in progress..." screen.
   - Serial Output:
     Each vote is also logged via the serial monitor, showing the candidate identifier and the updated vote count.
3. Reset Functionality:
   - Reset Button Pressed:
     When the reset button is pressed, the system:
     - Resets all candidate vote counts to zero.
     - Clears any ongoing election state.
     - Displays the message "Election Reset" briefly on the LCD before returning to the voting screen.
   - This allows the election process to start fresh if needed.

4.Displaying Final Results:

- Show Results Button Pressed:
  Once the voting period is over, pressing the show results button sets the election state to ended and triggers the final vote count display.
- LCD Display Update:
  The LCD is updated to show the overall vote counts for all candidates:
  - First Line: Displays the vote counts for Candidate 1 and Candidate 2 (e.g., "C1:3 C2:5")
  - Second Line: Displays the vote count for Candidate 3 (e.g., "C3:2")
- At this point, the system stops accepting additional votes, ensuring that the final result remains unchanged.

Conclusion

The electronic voting machine project successfully demonstrates a fully functional voting system that integrates an ESP32 microcontroller, a 16x2 I2C LCD display, and multiple push buttons. The design is centered around the following key achievements:

- Reliable Vote Counting:
  The system effectively handles candidate votes with built-in software debouncing to avoid erroneous multiple counts, ensuring that each button press is accurately registered as a single vote.
- User-Friendly Interface:
  The LCD provides clear and immediate visual feedback at every stage:
  - Initialization: Communicates system startup.
  - Voting: Confirms each vote with a temporary message.
  - Results Display: Clearly shows the final vote counts for all candidates.
- Robust Control Features:
  - Reset Functionality: Allows the election process to be restarted without any residual data from previous votes.
  - Final Results Lock-In: Ensures that, once the election is ended, no further votes can be registered, preserving the integrity of the outcome.
- Efficient Power Management:
  The entire project operates on a stable 3.3V supply, ensuring compatibility and energy efficiency across all components.
- Flexible and Scalable Design:
  The use of the ESP32 and the Arduino IDE allows for rapid development, easy debugging, and the potential for further enhancements, such as wireless communication or data logging to a remote server.

Overall, the project not only meets the functional requirements of an electronic voting machine but also serves as a robust foundation for further research and development in digital voting systems. The integration of hardware and software components, combined with real-time feedback and secure vote processing, demonstrates the feasibility and reliability of using modern microcontrollers in critical applications such as voting.

Electronic Voting Machine Review 1

Leveraging ESP32 for Modern Elections

22MID0190 Vignesh SB
22MID0189 Sanjay SM
22MID0197 Adit Ranganathan Suresh

Introduction

This presentation covers the essential aspects of electronic voting machines, focusing on the role of ESP32 technology in enhancing voting security and efficiency.

01

Overview

Definition of Electronic Voting Machines

Electronic voting machines (EVMs) are used to cast, count, and record votes digitally. These machines ensure accuracy and security during elections by minimizing human error and providing quick results.

Importance of ESP32 in Voting Systems

The ESP32 microcontroller provides Wi-Fi and Bluetooth capabilities, making it essential for secure communication and data transfer in voting systems. Its low power consumption and processing power contribute to reliable and efficient electronic voting.

Basic Functionality and Features

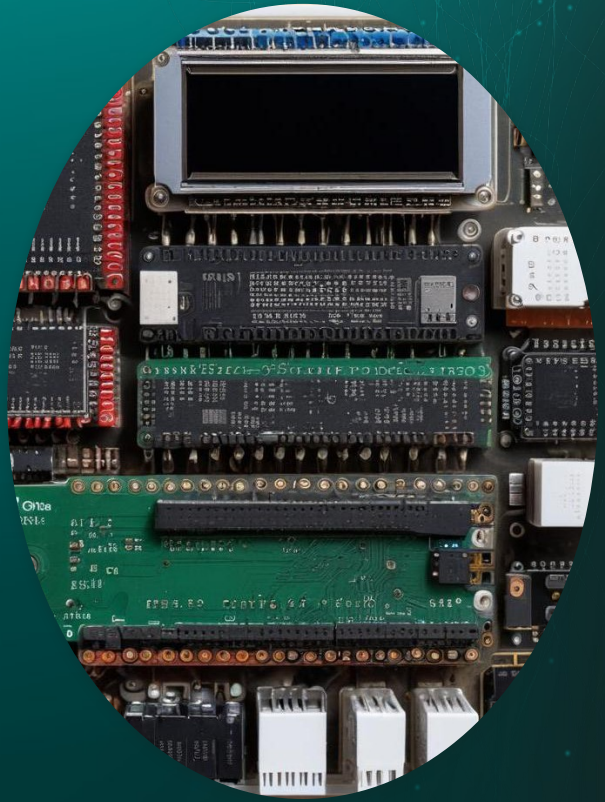
Electronic voting machines typically include a secure ballot display, touch interface or keypad for selection, and immediate result verification features. They ensure each vote is accurately captured while providing voter anonymity, with capabilities to monitor votes for irregularities.

02

Design

Components Required for ESP32 Voting Machine

Key components include the ESP32 microcontroller, buttons, a battery for power backup.



Circuit Design and Wiring

The circuit design involves connecting the ESP32 to the input devices buttons/, output devices (display). Proper wiring ensures minimal interference and reliable communication between components, highlighting safety protocols.

User Interface and User Experience

An effective user interface is crucial for voter accessibility. The design should be intuitive, providing clear instructions and feedback during the voting process. Engaging design also reduces voter errors and enhances overall satisfaction with the voting experience.

Conclusions

Incorporating ESP32 technology into electronic voting machines enhances security, reliability, and user experience. As this technology evolves, ensuring accessibility and voter trust remains paramount to the democratic process.

Thankyou